

AI ASSISTED CODING

ASSIGNMENT- 12.5

NAME: Varshitha. G

HTNO: 2303A51103

Task Description #1 (Sorting – Merge Sort Implementation)

- Task: Use AI to generate a Python program that implements the Merge Sort algorithm.
- Instructions:
 - o Prompt AI to create a function `merge_sort(arr)` that sorts a list in ascending order.
 - o Ask AI to include time complexity and space complexity in the function docstring.
 - o Verify the generated code with test cases.
- Expected Output:
 - o A functional Python script implementing Merge Sort with proper documentation.

```
2 def merge_sort(arr):
3     """
4     Sorts a list in ascending order using the merge sort algorithm.
5
6     Time Complexity: O(n log n) - where n is the number of elements in the list.
7     Space Complexity: O(n) - due to the temporary arrays used for merging.
8
9     Parameters:
10    arr (list): The list to be sorted.
11
12    Returns:
13    list: A new sorted list in ascending order.
14    """
15    if len(arr) <= 1:
16        | return arr
17
18    mid = len(arr) // 2
19    left_half = merge_sort(arr[:mid])
20    right_half = merge_sort(arr[mid:])
21
22    return merge(left_half, right_half)
23 def merge(left, right):
24     """
25     Merges two sorted lists into a single sorted list.
26     Parameters:
27     left (list): The first sorted list.
28     right (list): The second sorted list.
29     Returns:
30     list: A merged sorted list containing all elements from both input lists.
31     """
32     merged = []
33     left_index = right_index = 0
34
35     while left_index < len(left) and right_index < len(right):
36         | if left[left_index] < right[right_index]:
37         |     merged.append(left[left_index])
38         |     left_index += 1
39         | else:
40         |     merged.append(right[right_index])
41         |     right_index += 1
42
43     # If there are remaining elements in the left half, add them to merged
```

```
43 # If there are remaining elements in the left half, add them to merged
44 while left_index < len(left):
45     merged.append(left[left_index])
46     left_index += 1
47
48 # If there are remaining elements in the right half, add them to merged
49 while right_index < len(right):
50     merged.append(right[right_index])
51     right_index += 1
52
53 return merged
54 # Test cases
55 if __name__ == "__main__":
56     test_cases = [
57         [38, 27, 43, 3, 9, 82, 10],
58         [5, 2, 9, 1, 5, 6],
59         [12, 11, 13, 5, 6, 7],
60         [],
61         [1],
62         [2, 1]
63     ]
64
65     for i, test in enumerate(test_cases):
66         sorted_list = merge_sort(test)
67         print(f"Test case {i + 1}: {sorted_list}")
68
```

PROBLEMS	OUTPUT	TERMINAL	PORTS
> TERMINAL Test case 1: [3, 9, 10, 27, 38, 43, 82] Test case 2: [1, 2, 5, 5, 6, 9] Test case 3: [5, 6, 7, 11, 12, 13] Test case 4: [] Test case 5: [1] Test case 6: [1, 2] Test case 5: [1] Test case 6: [1, 2] Test case 6: [1, 2] PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>			

Task Description #2 (Searching – Binary Search with AI Optimization)

- Task: Use AI to create a binary search function that finds a target element in a sorted list.
- Instructions:
 - o Prompt AI to create a function `binary_search(arr, target)` returning the index of the target or -1 if not found.
 - o Include docstrings explaining best, average, and worst-case complexities.
 - o Test with various inputs.
- Expected Output:
 - o Python code implementing binary search with AI-generated comments and docstrings.

```
def binary_search(arr, target):  
    """  
    Performs binary search on a sorted list to find the index of the target element.  
    Time Complexity:  
    - Best Case:  $O(1)$  - when the target is at the middle of the list.  
    - Average Case:  $O(\log n)$  - where  $n$  is the number of elements in the list.  
    - Worst Case:  $O(\log n)$  - when the target is not present in the list or is at one of the ends.  
    Space Complexity:  $O(1)$  - as we are using a constant amount of space for variables.  
    Parameters:  
    arr (list): A sorted list in which to search for the target.  
    target: The element to search for.  
    Returns:  
    int: The index of the target if found, otherwise -1.  
    """  
    left, right = 0, len(arr) - 1  
    while left <= right:  
        mid = left + (right - left) // 2  
        # Check if the target is present at mid  
        if arr[mid] == target:  
            return mid  
        # If target is greater, ignore the left half  
        elif arr[mid] < target:  
            left = mid + 1  
        # If target is smaller, ignore the right half  
        else:  
            right = mid - 1  
    # Target was not found in the array  
    return -1  
# Test cases  
if __name__ == "__main__":  
    test_cases = [  
        ([1, 2, 3, 4, 5], 3), # Target is in the middle  
        ([1, 2, 3, 4, 5], 1), # Target is at the beginning  
        ([1, 2, 3, 4, 5], 5), # Target is at the end  
        ([1, 2, 3, 4, 5], 6), # Target is not present  
        ([], 1), # Empty list  
        ([1], 1), # Single element list where target is present  
        ([1], 0), # Single element list where target is not present  
    ]
```

```
100 if __name__ == "__main__":  
101     test_cases = [  
102         ([1, 2, 3, 4, 5], 3), # Target is in the middle  
103         ([1, 2, 3, 4, 5], 1), # Target is at the beginning  
104         ([1, 2, 3, 4, 5], 5), # Target is at the end  
105         ([1, 2, 3, 4, 5], 6), # Target is not present  
106         ([], 1), # Empty list  
107         ([1], 1), # Single element list where target is present  
108         ([1], 0), # Single element list where target is not present  
109     ]  
110  
111     for i, (arr, target) in enumerate(test_cases):  
112         result = binary_search(arr, target)  
113         print(f"Test case {i + 1}: Target {target} in {arr} -> Index: {result}")  
114
```

PROBLEMS	OUTPUT	TERMINAL	PORTS
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Programs/Python/Python310/python.exe C:\Users\varsh\OneDrive\ドキュメント\third\AIAC\main.py Test case 1: Target 3 in [1, 2, 3, 4, 5] -> Index: 2 Test case 2: Target 1 in [1, 2, 3, 4, 5] -> Index: 0 Test case 3: Target 5 in [1, 2, 3, 4, 5] -> Index: 4 Test case 4: Target 6 in [1, 2, 3, 4, 5] -> Index: -1 Test case 5: Target 1 in [] -> Index: -1 Test case 6: Target 1 in [1] -> Index: 0 Test case 7: Target 0 in [1] -> Index: -1 PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>			

Task Description #3: Smart Healthcare Appointment Scheduling System

A healthcare platform maintains appointment records containing appointment ID, patient name, doctor name, appointment time, and consultation fee. The system needs to:

1. Search appointments using appointment ID.
2. Sort appointments based on time or consultation fee.

Student Task

- Use AI to recommend suitable searching and sorting algorithms.
- Justify the selected algorithms.
- Implement the algorithms in Python.

```
def binary_search(arr, target):  
    """  
    Performs binary search on a sorted list to find the index of the target element.  
    Time Complexity:  $O(\log n)$  - where  $n$  is the number of elements in the list.  
    Space Complexity:  $O(1)$  - as we are using a constant amount of space for variables.  
    Parameters:  
    arr (list): A sorted list in which to search for the target.  
    target: The element to search for.  
    Returns:  
    int: The index of the target if found, otherwise -1.  
    """  
    left, right = 0, len(arr) - 1  
    while left <= right:  
        mid = left + (right - left) // 2  
        if arr[mid]['appointment_id'] == target:  
            return mid  
        elif arr[mid]['appointment_id'] < target:  
            left = mid + 1  
        else:  
            right = mid - 1  
    return -1  
  
def merge_sort(arr, key):  
    """  
    Sorts a list of dictionaries based on a specified key using the merge sort algorithm.  
    Time Complexity:  $O(n \log n)$  - where  $n$  is the number of elements in the list.  
    Space Complexity:  $O(n)$  - due to the temporary arrays used for merging.  
    Parameters:  
    arr (list): The list of dictionaries to be sorted.  
    key (str): The key in the dictionary to sort by.  
    Returns:  
    list: A new sorted list of dictionaries based on the specified key.  
    """  
    if len(arr) <= 1:  
        return arr  
    mid = len(arr) // 2
```

```

    left_half = merge_sort(arr[:mid], key)
    right_half = merge_sort(arr[mid:], key)
    return merge(left_half, right_half, key)

def merge(left, right, key):
    """Merges two sorted lists of dictionaries into a single sorted list based on a specified key.
    Parameters:
    left (list): The first sorted list of dictionaries.
    right (list): The second sorted list of dictionaries.
    key (str): The key in the dictionary to sort by.
    Returns:
    list: A merged sorted list of dictionaries based on the specified key.
    """
    merged = []
    left_index = right_index = 0
    while left_index < len(left) and right_index < len(right):
        if left[left_index][key] < right[right_index][key]:
            merged.append(left[left_index])
            left_index += 1
        else:
            merged.append(right[right_index])
            right_index += 1
    while left_index < len(left):
        merged.append(left[left_index])
        left_index += 1
    while right_index < len(right):
        merged.append(right[right_index])
        right_index += 1
    return merged

```

```

if __name__ == "__main__":
    appointments = [
        {'appointment_id': 1, 'patient_name': 'Alice', 'doctor_name': 'Dr. Smith', 'appointment_time': '2024-07-01 10:00', 'consultation_fee': 100},
        {'appointment_id': 2, 'patient_name': 'Bob', 'doctor_name': 'Dr. Jones', 'appointment_time': '2024-07-01 11:00', 'consultation_fee': 150},
        {'appointment_id': 3, 'patient_name': 'Charlie', 'doctor_name': 'Dr. Brown', 'appointment_time': '2024-07-01 09:00', 'consultation_fee': 120}
    ]
    # Sort appointments by appointment time
    sorted_by_time = merge_sort(appointments, key='appointment_time')
    print("Appointments sorted by time:")
    for appointment in sorted_by_time:
        print(appointment)
    # Sort appointments by consultation fee
    sorted_by_fee = merge_sort(appointments, key='consultation_fee')
    print("\nAppointments sorted by consultation fee:")
    for appointment in sorted_by_fee:
        print(appointment)
    # Search for an appointment by ID
    target_id = 2
    index = binary_search(sorted_by_time, target_id)
    if index != -1:
        print(f"\nAppointment with ID {target_id} found: {sorted_by_time[index]}")
    else:
        print(f"\nAppointment with ID {target_id} not found.")

```

```

ive/ドキュメント/third/AIAC/lab_assignment_12.5.py
Appointments sorted by time:
{'appointment_id': 3, 'patient_name': 'Charlie', 'doctor_name': 'Dr. Brown', 'appointment_time': '2024-07-01 09:00', 'consultation_fee': 120}
{'appointment_id': 1, 'patient_name': 'Alice', 'doctor_name': 'Dr. Smith', 'appointment_time': '2024-07-01 10:00', 'consultation_fee': 100}
{'appointment_id': 2, 'patient_name': 'Bob', 'doctor_name': 'Dr. Jones', 'appointment_time': '2024-07-01 11:00', 'consultation_fee': 150}

Appointments sorted by consultation fee:
{'appointment_id': 1, 'patient_name': 'Alice', 'doctor_name': 'Dr. Smith', 'appointment_time': '2024-07-01 10:00', 'consultation_fee': 100}
{'appointment_id': 3, 'patient_name': 'Charlie', 'doctor_name': 'Dr. Brown', 'appointment_time': '2024-07-01 09:00', 'consultation_fee': 120}
{'appointment_id': 2, 'patient_name': 'Bob', 'doctor_name': 'Dr. Jones', 'appointment_time': '2024-07-01 11:00', 'consultation_fee': 150}

Appointment with ID 2 found: {'appointment_id': 2, 'patient_name': 'Bob', 'doctor_name': 'Dr. Jones', 'appointment_time': '2024-07-01 11:00', 'consultation_fee': 150}

```

For searching by appointment ID, a hash table lookup is best since it gives instant access using the unique ID. For sorting by time or consultation fee, merge sort is recommended because it's efficient ($O(n \log n)$) and stable, preserving the order of equal values.

Task Description #4: Railway Ticket Reservation System Scenario A railway reservation system stores booking details such as ticket ID, passenger name, train number, seat number, and travel date. The system must:

1. Search tickets using ticket ID.
2. Sort bookings based on travel date or seat number.

Student Task

- Identify efficient algorithms using AI assistance.
- Justify the algorithm choices.
- Implement searching and sorting in Python.

```
6 def binary_search(arr, target):
7     """
8     Performs binary search on a sorted list to find the index of the target element.
9     Time Complexity: O(log n) - where n is the number of elements in the list.
10    Space Complexity: O(1) - as we are using a constant amount of space for variables.
11    Parameters:
12    arr (list): A sorted list in which to search for the target.
13    target: The element to search for.
14    Returns:
15    int: The index of the target if found, otherwise -1.
16    """
17    left, right = 0, len(arr) - 1
18    while left <= right:
19        mid = left + (right - left) // 2
20        if arr[mid]['ticket_id'] == target:
21            return mid
22        elif arr[mid]['ticket_id'] < target:
23            left = mid + 1
24        else:
25            right = mid - 1
26    return -1
27
28 def merge_sort(arr, key):
29     """
30     Sorts a list of dictionaries based on a specified key using the merge sort algorithm.
31     Time Complexity: O(n log n) - where n is the number of elements in the list
32     Space Complexity: O(n) - due to the temporary arrays used for merging.
33     Parameters:
34     arr (list): The list of dictionaries to be sorted.
35     key (str): The key in the dictionary to sort by.
36     Returns:
37     list: A new sorted list of dictionaries based on the specified key.
38     """
39    if len(arr) <= 1:
40        return arr
41    mid = len(arr) // 2
42    left_half = merge_sort(arr[:mid], key)
43    right_half = merge_sort(arr[mid:], key)
44    return merge(left_half, right_half, key)
```

```
289 # Example usage
290 if __name__ == "__main__":
291     bookings = [
292         {'ticket_id': 1, 'passenger_name': 'Alice', 'train_number': '123A', 'seat_number': '12B', 'travel_date': '2024-08-01'},
293         {'ticket_id': 2, 'passenger_name': 'Bob', 'train_number': '456B', 'seat_number': '15C', 'travel_date': '2024-08-02'},
294         {'ticket_id': 3, 'passenger_name': 'Charlie', 'train_number': '789C', 'seat_number': '10A', 'travel_date': '2024-08-01'}
295     ]
296     # Sort bookings by travel date
297     sorted_by_date = merge_sort(bookings, key='travel_date')
298     print("Bookings sorted by travel date:")
299     for booking in sorted_by_date:
300         print(booking)
301     # Sort bookings by seat number
302     sorted_by_seat = merge_sort(bookings, key='seat_number')
303     print("\nBookings sorted by seat number:")
304     for booking in sorted_by_seat:
305         print(booking)
306     # Search for a booking by ticket ID
307     target_id = 2
308     index = binary_search(sorted_by_date, target_id)
309     if index != -1:
310         print(f"\nBooking with ticket ID {target_id} found: {sorted_by_date[index]}")
311     else:
312         print(f"\nBooking with ticket ID {target_id} not found.")
```

PROBLEMS OUTPUT **TERMINAL** PORTS

Python: lab_assignment_12.5

```
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Programs/Python/Python313/python.exe c:/Users/varsh/OneDrive/ドキュメント\third\AIAC/lab_assignment_12.5.py
Bookings sorted by travel date:
{'ticket_id': 3, 'passenger_name': 'Charlie', 'train_number': '789C', 'seat_number': '10A', 'travel_date': '2024-08-01'}
{'ticket_id': 3, 'passenger_name': 'Charlie', 'train_number': '789C', 'seat_number': '10A', 'travel_date': '2024-08-01'}
{'ticket_id': 1, 'passenger_name': 'Alice', 'train_number': '123A', 'seat_number': '12B', 'travel_date': '2024-08-01'}
{'ticket_id': 2, 'passenger_name': 'Bob', 'train_number': '456B', 'seat_number': '15C', 'travel_date': '2024-08-02'}

Bookings sorted by seat number:
{'ticket_id': 3, 'passenger_name': 'Charlie', 'train_number': '789C', 'seat_number': '10A', 'travel_date': '2024-08-01'}
{'ticket_id': 1, 'passenger_name': 'Alice', 'train_number': '123A', 'seat_number': '12B', 'travel_date': '2024-08-01'}
{'ticket_id': 1, 'passenger_name': 'Alice', 'train_number': '123A', 'seat_number': '12B', 'travel_date': '2024-08-01'}
{'ticket_id': 2, 'passenger_name': 'Bob', 'train_number': '456B', 'seat_number': '15C', 'travel_date': '2024-08-02'}

○ Booking with ticket ID 2 found: {'ticket_id': 2, 'passenger_name': 'Bob', 'train_number': '456B', 'seat_number': '15C', 'travel_date': '2024-08-02'}
```

Task Description #5: Smart Hostel Room Allocation System

A hostel management system stores student room allocation details including student ID, room number, floor, and allocation date. The system needs to:

1. Search allocation details using student ID.
2. Sort records based on room number or allocation date.

Student Task

- Use AI to suggest optimized algorithms.
- Justify the selections.
- Implement the solution in Python.

```

29 def binary_search(arr, target):
30     """
31     Performs binary search on a sorted list to find the index of the target element.
32     Time Complexity:  $O(\log n)$  - where  $n$  is the number of elements in the list.
33     Space Complexity:  $O(1)$  - as we are using a constant amount of space for variables.
34     Parameters:
35     arr (list): A sorted list in which to search for the target.
36     target: The element to search for.
37     Returns:
38     int: The index of the target if found, otherwise -1.
39     """
40     left, right = 0, len(arr) - 1
41     while left <= right:
42         mid = left + (right - left) // 2
43         if arr[mid]['student_id'] == target:
44             return mid
45         elif arr[mid]['student_id'] < target:
46             left = mid + 1
47         else:
48             right = mid - 1
49     return -1
50
51 def merge_sort(arr, key):
52     """ Sorts a list of dictionaries based on a specified key using the merge sort algorithm.
53     Time Complexity:  $O(n \log n)$  - where  $n$  is the number of elements in the list.
54     Space Complexity:  $O(n)$  - due to the temporary arrays used for merging.
55     Parameters:
56     arr (list): The list of dictionaries to be sorted.
57     key (str): The key in the dictionary to sort by.
58     Returns:
59     list: A new sorted list of dictionaries based on the specified key.
60     """
61     if len(arr) <= 1:
62         return arr
63     mid = len(arr) // 2
64     left_half = merge_sort(arr[:mid], key)
65     right_half = merge_sort(arr[mid:], key)
66     return merge(left_half, right_half, key)

```

```

def merge(left, right, key):
    """Merges two sorted lists of dictionaries into a single sorted list based on a specified key.
    Parameters:
    left (list): The first sorted list of dictionaries.
    right (list): The second sorted list of dictionaries.
    key (str): The key in the dictionary to sort by.
    Returns:
    list: A merged sorted list of dictionaries based on the specified key.
    """
    merged = []
    left_index = right_index = 0
    while left_index < len(left) and right_index < len(right):
        if left[left_index][key] < right[right_index][key]:
            merged.append(left[left_index])
            left_index += 1
        else:
            merged.append(right[right_index])
            right_index += 1
    while left_index < len(left):
        merged.append(left[left_index])
        left_index += 1
    while right_index < len(right):
        merged.append(right[right_index])
        right_index += 1
    return merged

# Example usage
if __name__ == "__main__":
    allocations = [
        {'student_id': 1, 'room_number': '101A', 'floor': 1, 'allocation_date': '2024-09-01'},
        {'student_id': 2, 'room_number': '102B', 'floor': 1, 'allocation_date': '2024-09-02'},
        {'student_id': 3, 'room_number': '201A', 'floor': 2, 'allocation_date': '2024-09-01'}
    ]
    # Sort records by room number
    sorted_by_room = merge_sort(allocations, key='room_number')
    print("Allocations sorted by room number:")
    for allocation in sorted_by_room:
        print(allocation)

```



```
397     ]
398     # Sort records by room number
399     sorted_by_room = merge_sort(allocations, key='room_number')
400     print("Allocations sorted by room number:")
401     for allocation in sorted_by_room:
402         print(allocation)
403     # Sort records by allocation date
404     sorted_by_date = merge_sort(allocations, key='allocation_date')
405     print("\nAllocations sorted by allocation date:")
406     for allocation in sorted_by_date:
407         print(allocation)
408     # Search for an allocation by student ID
409     target_id = 2
410     index = binary_search(sorted_by_room, target_id)
411     if index != -1:
412         print(f"\nAllocation with student ID {target_id} found: {sorted_by_room[index]}")
413     else:
414         print(f"\nAllocation with student ID {target_id} not found.")
415
```

PROBLEMS OUTPUT **TERMINAL** PORTS

▼ **TERMINAL** Python

```
Allocations sorted by allocation date:
{'student_id': 3, 'room_number': '201A', 'floor': 2, 'allocation_date': '2024-09-01'}
{'student_id': 1, 'room_number': '101A', 'floor': 1, 'allocation_date': '2024-09-01'}
{'student_id': 2, 'room_number': '102B', 'floor': 1, 'allocation_date': '2024-09-02'}

Allocations sorted by allocation date:
{'student_id': 3, 'room_number': '201A', 'floor': 2, 'allocation_date': '2024-09-01'}
{'student_id': 1, 'room_number': '101A', 'floor': 1, 'allocation_date': '2024-09-01'}
{'student_id': 2, 'room_number': '102B', 'floor': 1, 'allocation_date': '2024-09-02'}

{'student_id': 1, 'room_number': '101A', 'floor': 1, 'allocation_date': '2024-09-01'}
{'student_id': 2, 'room_number': '102B', 'floor': 1, 'allocation_date': '2024-09-02'}

Allocation with student ID 2 found: {'student_id': 2, 'room_number': '102B', 'floor': 1, 'allocation_date': '2024-09-02'}
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>
```

Task Description #6: Online Movie Streaming Platform

A streaming service maintains movie records with movie ID, title, genre, rating, and release year. The platform needs to:

1. Search movies by movie ID.
2. Sort movies based on rating or release year.

Student Task

- Recommend searching and sorting algorithms using AI.
- Justify the chosen algorithms.
- Implement Python functions.

```

def binary_search(arr, target):
    """
    Performs binary search on a sorted list to find the index of the target element.
    Time Complexity:  $O(\log n)$  - where  $n$  is the number of elements in the list.
    Space Complexity:  $O(1)$  - as we are using a constant amount of space for variables.
    Parameters:
    arr (list): A sorted list in which to search for the target.
    target: The element to search for.
    Returns:
    int: The index of the target if found, otherwise -1.
    """
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = left + (right - left) // 2
        if arr[mid]['movie_id'] == target:
            return mid
        elif arr[mid]['movie_id'] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1

def merge_sort(arr, key):
    """ Sorts a list of dictionaries based on a specified key using the merge sort algorithm.
    Time Complexity:  $O(n \log n)$  - where  $n$  is the number of elements in the list.
    Space Complexity:  $O(n)$  - due to the temporary arrays used for merging.
    Parameters:
    arr (list): The list of dictionaries to be sorted.
    key (str): The key in the dictionary to sort by.
    Returns:
    list: A new sorted list of dictionaries based on the specified key.
    """
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid], key)
    right_half = merge_sort(arr[mid:], key)
    return merge(left_half, right_half, key)

def merge(left, right, key):
    """Merges two sorted lists of dictionaries into a single sorted list based on a specified key.
    Parameters:
    left (list): The first sorted list of dictionaries.
    right (list): The second sorted list of dictionaries.
    key (str): The key in the dictionary to sort by.
    """

```

```

76
77     merged = []
78     left_index = right_index = 0
79     while left_index < len(left) and right_index < len(right):
80         if left[left_index][key] < right[right_index][key]:
81             merged.append(left[left_index])
82             left_index += 1
83         else:
84             merged.append(right[right_index])
85             right_index += 1
86     while left_index < len(left):
87         merged.append(left[left_index])
88         left_index += 1
89     while right_index < len(right):
90         merged.append(right[right_index])
91         right_index += 1
92     return merged
93
94 # Example usage
95 if __name__ == "__main__":
96     movies = [
97         {'movie_id': 1, 'title': 'Movie A', 'genre': 'Action', 'rating': 4.5, 'release_year': 2020},
98         {'movie_id': 2, 'title': 'Movie B', 'genre': 'Comedy', 'rating': 4.0, 'release_year': 2019},
99         {'movie_id': 3, 'title': 'Movie C', 'genre': 'Drama', 'rating': 4.8, 'release_year': 2021}
100     ]
101     # Sort movies by rating
102     sorted_by_rating = merge_sort(movies, key='rating')
103     print("Movies sorted by rating:")
104     for movie in sorted_by_rating:
105         print(movie)
106     # Sort movies by release year
107     sorted_by_year = merge_sort(movies, key='release_year')
108     print("\nMovies sorted by release year:")
109     for movie in sorted_by_year:
110         print(movie)
111     # Search for a movie by ID
112     target_id = 2
113     index = binary_search(sorted_by_rating, target_id)
114     if index != -1:
115         print(f"\nMovie with ID {target_id} found: {sorted_by_rating[index]}")
116     else:
117         print(f"\nMovie with ID {target_id} not found.")

```

```

ers/varsh/OneDrive/ドキュメント/third/AIAC/lab_assignment_12.5.py
Movies sorted by rating:
{'movie_id': 2, 'title': 'Movie B', 'genre': 'Comedy', 'rating': 4.0, 'release_year': 2019}
{'movie_id': 1, 'title': 'Movie A', 'genre': 'Action', 'rating': 4.5, 'release_year': 2020}
{'movie_id': 3, 'title': 'Movie C', 'genre': 'Drama', 'rating': 4.8, 'release_year': 2021}

Movies sorted by release year:
{'movie_id': 2, 'title': 'Movie B', 'genre': 'Comedy', 'rating': 4.0, 'release_year': 2019}
{'movie_id': 1, 'title': 'Movie A', 'genre': 'Action', 'rating': 4.5, 'release_year': 2020}
{'movie_id': 3, 'title': 'Movie C', 'genre': 'Drama', 'rating': 4.8, 'release_year': 2021}

Movie with ID 2 not found.
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>

```

Task Description #7: Smart Agriculture Crop Monitoring System

An agriculture monitoring system stores crop data with crop ID, crop name, soil moisture level, temperature, and yield estimate. Farmers need to:

1. Search crop details using crop ID.
2. Sort crops based on moisture level or yield estimate.

Student Task• Use AI-assisted reasoning to select algorithms. • Justify algorithm suitability.

- Implement searching and sorting in Python.

```

3 def binary_search(arr, target):
4     """
5     Performs binary search on a sorted list to find the index of the target element.
6     Time Complexity:  $O(\log n)$  - where  $n$  is the number of elements in the list.
7     Space Complexity:  $O(1)$  - as we are using a constant amount of space for variables.
8     Parameters:
9     arr (list): A sorted list in which to search for the target.
10    target: The element to search for.
11    Returns:
12    int: The index of the target if found, otherwise -1.
13    """
14    left, right = 0, len(arr) - 1
15    while left <= right:
16        mid = left + (right - left) // 2
17        if arr[mid]['crop_id'] == target:
18            return mid
19        elif arr[mid]['crop_id'] < target:
20            left = mid + 1
21        else:
22            right = mid - 1
23    return -1
24
25 def merge_sort(arr, key):
26     """ Sorts a list of dictionaries based on a specified key using the merge sort algorithm.
27     Time Complexity:  $O(n \log n)$  - where  $n$  is the number of elements in the list.
28     Space Complexity:  $O(n)$  - due to the temporary arrays used for merging.
29     Parameters:
30     arr (list): The list of dictionaries to be sorted.
31     key (str): The key in the dictionary to sort by.
32     Returns:
33     list: A new sorted list of dictionaries based on the specified key.
34     """
35     if len(arr) <= 1:
36         return arr
37     mid = len(arr) // 2
38     left_half = merge_sort(arr[:mid], key)
39     right_half = merge_sort(arr[mid:], key)
40     return merge(left_half, right_half, key)

```

```

1 def merge(left, right, key):
2     """Merges two sorted lists of dictionaries into a single sorted list based on a specified key.
3     Parameters:
4     left (list): The first sorted list of dictionaries.
5     right (list): The second sorted list of dictionaries.
6     key (str): The key in the dictionary to sort by.
7     Returns:
8     list: A merged sorted list of dictionaries based on the specified key.
9     """
10    merged = []
11    left_index = right_index = 0
12    while left_index < len(left) and right_index < len(right):
13        if left[left_index][key] < right[right_index][key]:
14            merged.append(left[left_index])
15            left_index += 1
16        else:
17            merged.append(right[right_index])
18            right_index += 1
19    while left_index < len(left):
20        merged.append(left[left_index])
21        left_index += 1
22    while right_index < len(right):
23        merged.append(right[right_index])
24        right_index += 1
25    return merged
26
27 # Example usage
28 if __name__ == "__main__":
29     crops = [
30         {'crop_id': 1, 'crop_name': 'Wheat', 'soil_moisture_level': 30, 'temperature': 25, 'yield_estimate': 1000},
31         {'crop_id': 2, 'crop_name': 'Corn', 'soil_moisture_level': 40, 'temperature': 28, 'yield_estimate': 1500},
32         {'crop_id': 3, 'crop_name': 'Rice', 'soil_moisture_level': 50, 'temperature': 30, 'yield_estimate': 2000}
33     ]
34     # Sort crops by soil moisture level
35     sorted_by_moisture = merge_sort(crops, key='soil_moisture_level')
36     print("Crops sorted by soil moisture level:")
37     for crop in sorted_by_moisture:
38         print(crop)
39     # Sort crops by yield estimate
40     sorted_by_yield = merge_sort(crops, key='yield_estimate')
41     print("\nCrops sorted by yield estimate:")

```

```
599 if __name__ == "__main__":
600     crops = [
601         {'crop_id': 1, 'crop_name': 'Wheat', 'soil_moisture_level': 30, 'temperature': 25, 'yield_estimate': 1000}
602         {'crop_id': 2, 'crop_name': 'Corn', 'soil_moisture_level': 40, 'temperature': 28, 'yield_estimate': 1500}
603         {'crop_id': 3, 'crop_name': 'Rice', 'soil_moisture_level': 50, 'temperature': 30, 'yield_estimate': 2000}
604     ]
605     # Sort crops by soil moisture level
606     sorted_by_moisture = merge_sort(crops, key='soil_moisture_level')
607     print("Crops sorted by soil moisture level:")
608     for crop in sorted_by_moisture:
609         print(crop)
610     # Sort crops by yield estimate
611     sorted_by_yield = merge_sort(crops, key='yield_estimate')
612     print("\nCrops sorted by yield estimate:")
613     for crop in sorted_by_yield:
614         print(crop)
615     # Search for a crop by ID
616     target_id = 2
617     index = binary_search(sorted_by_moisture, target_id)
618     if index != -1:
619         print(f"\nCrop with ID {target_id} found: {sorted_by_moisture[index]}")
620     else:
621         print(f"\nCrop with ID {target_id} not found.")
622
```

PROBLEMS OUTPUT TERMINAL PORTS

Python: lab_assignment_12.5

o Crops sorted by soil moisture level:

```
{'crop_id': 1, 'crop_name': 'Wheat', 'soil_moisture_level': 30, 'temperature': 25, 'yield_estimate': 1000}
{'crop_id': 2, 'crop_name': 'Corn', 'soil_moisture_level': 40, 'temperature': 28, 'yield_estimate': 1500}
{'crop_id': 3, 'crop_name': 'Rice', 'soil_moisture_level': 50, 'temperature': 30, 'yield_estimate': 2000}
```

Crops sorted by yield estimate:

```
{'crop_id': 1, 'crop_name': 'Wheat', 'soil_moisture_level': 30, 'temperature': 25, 'yield_estimate': 1000}
{'crop_id': 2, 'crop_name': 'Corn', 'soil_moisture_level': 40, 'temperature': 28, 'yield_estimate': 1500}
{'crop_id': 3, 'crop_name': 'Rice', 'soil_moisture_level': 50, 'temperature': 30, 'yield_estimate': 2000}
```

Crop with ID 2 found: {'crop_id': 2, 'crop_name': 'Corn', 'soil_moisture_level': 40, 'temperature': 28, 'yield_estimate': 1500}

PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>

Task Description #8: Airport Flight Management System

An airport system stores flight information including flight ID, airline name, departure time, arrival time, and status. The system must:

1. Search flight details using flight ID.
2. Sort flights based on departure time or arrival time.

Student Task

- Use AI to recommend algorithms.
- Justify the algorithm selection.
- Implement searching and sorting logic in Python.

```
def binary_search(arr, target):
    """
    Performs binary search on a sorted list to find the index of the target element.
    Time Complexity: O(log n) - where n is the number of elements in the list.
    Space Complexity: O(1) - as we are using a constant amount of space for variables.
    Parameters:
    arr (list): A sorted list in which to search for the target.
    target: The element to search for.
    Returns:
    int: The index of the target if found, otherwise -1.
    """
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = left + (right - left) // 2
        if arr[mid]['flight_id'] == target:
            return mid
        elif arr[mid]['flight_id'] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1

def merge_sort(arr, key):
    """ Sorts a list of dictionaries based on a specified key using the merge sort algorithm.
    Time Complexity: O(n log n) - where n is the number of elements in the list.
    Space Complexity: O(n) - due to the temporary arrays used for merging.
    Parameters:
    arr (list): The list of dictionaries to be sorted.
    key (str): The key in the dictionary to sort by.
    Returns:
    list: A new sorted list of dictionaries based on the specified key.
    """
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid], key)
    right_half = merge_sort(arr[mid:], key)
    return merge(left_half, right_half, key)
```

```
def merge(left, right, key):
    """Merges two sorted lists of dictionaries into a single sorted list based on a specified key.
    Parameters:
    left (list): The first sorted list of dictionaries.
    right (list): The second sorted list of dictionaries.
    key (str): The key in the dictionary to sort by.
    Returns:
    list: A merged sorted list of dictionaries based on the specified key.
    """
    merged = []
    left_index = right_index = 0
    while left_index < len(left) and right_index < len(right):
        if left[left_index][key] < right[right_index][key]:
            merged.append(left[left_index])
            left_index += 1
        else:
            merged.append(right[right_index])
            right_index += 1
    while left_index < len(left):
        merged.append(left[left_index])
        left_index += 1
    while right_index < len(right):
        merged.append(right[right_index])
        right_index += 1
    return merged

# Example usage
if __name__ == "__main__":
    flights = [
        {'flight_id': 1, 'airline_name': 'Air A', 'departure_time': '2024-10-01 08:00', 'arrival_time': '2024-10-01 10:00'},
        {'flight_id': 2, 'airline_name': 'Air B', 'departure_time': '2024-10-01 09:00', 'arrival_time': '2024-10-01 11:00'},
        {'flight_id': 3, 'airline_name': 'Air C', 'departure_time': '2024-10-01 07:00', 'arrival_time': '2024-10-01 09:00'}
    ]
    # Sort flights by departure time
    sorted_by_departure = merge_sort(flights, key='departure_time')
    print("Flights sorted by departure time:")
    for flight in sorted_by_departure:
        print(flight)
    # Sort flights by arrival time
    sorted_by_arrival = merge_sort(flights, key='arrival_time')
    print("\nFlights sorted by arrival time:")
    for flight in sorted_by_arrival:
        print(flight)
```

```

    {'flight_id': 1, 'airline_name': 'Air A', 'departure_time': '2024-10-01 08:00', 'arrival_time': '2024-10-01 10:00', 'status': 'On Time'}
    {'flight_id': 2, 'airline_name': 'Air B', 'departure_time': '2024-10-01 09:00', 'arrival_time': '2024-10-01 11:00', 'status': 'Delayed'}
    {'flight_id': 3, 'airline_name': 'Air C', 'departure_time': '2024-10-01 07:00', 'arrival_time': '2024-10-01 09:00', 'status': 'On Time'}
]

# Sort flights by departure time
sorted_by_departure = merge_sort(flights, key='departure_time')
print("Flights sorted by departure time:")
for flight in sorted_by_departure:
    print(flight)

# Sort flights by arrival time
sorted_by_arrival = merge_sort(flights, key='arrival_time')
print("\nFlights sorted by arrival time:")
for flight in sorted_by_arrival:
    print(flight)

# Search for a flight by ID
target_id = 2
index = binary_search(sorted_by_departure, target_id)
if index != -1:
    print(f"\nFlight with ID {target_id} found: {sorted_by_departure[index]}")
else:
    print(f"\nFlight with ID {target_id} not found.")

```

BLEMS OUTPUT **TERMINAL** PORTS

TERMINAL

Python: lab_assignment_12.5 + v

```

Flights sorted by departure time:
{'flight_id': 3, 'airline_name': 'Air C', 'departure_time': '2024-10-01 07:00', 'arrival_time': '2024-10-01 09:00', 'status': 'On Time'}
{'flight_id': 1, 'airline_name': 'Air A', 'departure_time': '2024-10-01 08:00', 'arrival_time': '2024-10-01 10:00', 'status': 'On Time'}
{'flight_id': 2, 'airline_name': 'Air B', 'departure_time': '2024-10-01 09:00', 'arrival_time': '2024-10-01 11:00', 'status': 'Delayed'}

Flights sorted by arrival time:
{'flight_id': 3, 'airline_name': 'Air C', 'departure_time': '2024-10-01 07:00', 'arrival_time': '2024-10-01 09:00', 'status': 'On Time'}
{'flight_id': 1, 'airline_name': 'Air A', 'departure_time': '2024-10-01 08:00', 'arrival_time': '2024-10-01 10:00', 'status': 'On Time'}
{'flight_id': 2, 'airline_name': 'Air B', 'departure_time': '2024-10-01 09:00', 'arrival_time': '2024-10-01 11:00', 'status': 'Delayed'}

Flight with ID 2 found: {'flight_id': 2, 'airline_name': 'Air B', 'departure_time': '2024-10-01 09:00', 'arrival_time': '2024-10-01 11:00', 'status': 'Delayed'}

```